Boolean forms, timers, LFSRs and Conway

The final connected series: K-maps, Rules 90 and 110, timer control, three LFSRs and Conway. End with an LFSR definitions and revision reference.

Kapil Tripathi | Verilog and digital design | 45 pages

CONTENTS / click a topic to jump

Reading days group the explanations, including later revisits. Tracker day labels and original dated submission evidence remain unchanged.

Use the linked contents or PDF bookmarks. Page numbers match the PDF viewer. Source questions and technical evidence remain beside their explanations.

Questions answered

Questions answered (continued)

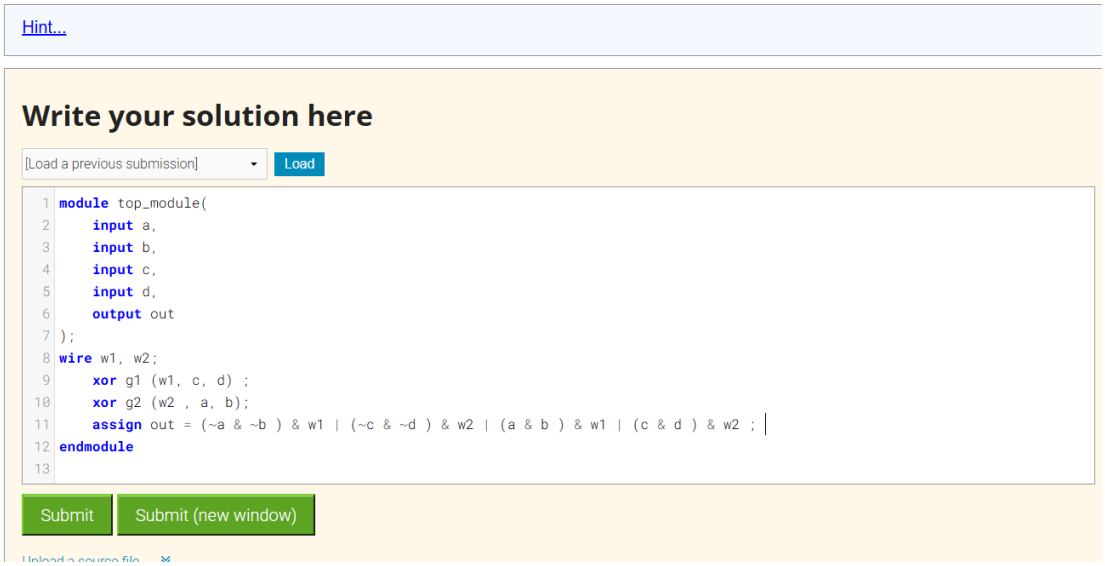
ENTRY 161 / KMAP4

Why the K-map expression uses |, not +

The four bracketed expressions are alternative one-bit conditions. The output must be 1 when any condition is true, so the combining operation is OR.

On paper, Boolean algebra often writes + for OR. In Verilog, the symbols have programming-language meanings: | is bitwise OR, while + is arithmetic addition.

Your accepted construction




```
wire w1, w2;
xor g1 (w1, c, d);
xor g2 (w2, a, b);
assign out = (~a & ~b) & w1 | (~c & ~d) & w2
           | ( a &  b) & w1 | ( c &  d) & w2;
```

Read it aloud as: first condition **OR** second condition **OR** third condition **OR** fourth condition. Parentheses would make the grouping easier to scan, although Verilog already evaluates & before |.

Source: HDLBits Kmap4. The saved submission evidence shows **Status: Success!**.

ENTRY 161 / OPERATOR CHECK

Why + can appear to work here

Arithmetic and OR differ as soon as two true terms overlap. This particular expression hides that difference because its four product terms are mutually exclusive.

Inputs	Bitwise OR	Arithmetic addition
0 and 0	0 | 0 = 0	0 + 0 = 0
0 and 1	0 | 1 = 1	0 + 1 = 1
1 and 1	1 | 1 = 1	1 + 1 = binary 10

The carry is the warning

```
wire term1 = 1'b1;
wire term2 = 1'b1;
wire       out_or = term1 | term2;                 // 1
wire [1:0] sum    = {1'b0,term1} + {1'b0,term2};  // 2'b10
```

If an arithmetic result is kept in only one bit, its carry can be discarded. That can turn `1 + 1` into a stored 0, while OR must remain 1. The explicit two-bit operands above expose the carry instead of relying on expression-size rules.

Why the submitted terms never overlap

`w1 = c ^ d` is 1 only when `c,d` differ. Its two terms then select either `a,b = 00` or `a,b = 11`. Similarly, `w2 = a ^ b` is 1 only when `a,b` differ, and its terms require `c,d = 00` or `c,d = 11`. Those cases cannot be true together, so at most one product term is 1.

What the exhaustive check proves

All 16 input combinations were checked. The submitted expression equals `a ^ b ^ c ^ d`, and never has two active product terms. Therefore + could accidentally match for this exact expression - but | is still the correct, intention-revealing operator.

ENTRY 164 / BOOLEAN FORMS

SOP, POS and De Morgan's law

SOP and POS are two ways to implement the same Boolean function. The difference is the form used to describe it, not a different output.

Form	Structure	K-map route
SOP	OR of product terms; each product term is an AND of literals.	Group output-1 cells, write one product term per group, then OR the terms.
POS	AND of sum terms; each sum term is an OR of literals.	Group output-0 cells, write one sum term per group, then AND the terms.

A precise way to remember them

For SOP, each 1-group creates a condition that makes the function true. For POS, each 0-group creates a sum term that becomes false on that group; ANDing the sum terms blocks the zero cases. Both final expressions still calculate the same output.

Correction to the earlier short note: saying 'SOP describes when the output is 1 and POS describes when it is 0' was too loose. The accurate statement is that the derivation starts from 1-cells for SOP and 0-cells for POS.

De Morgan's law

```
~(A & B) = (~A) | (~B)
~(A | B) = (~A) & (~B)
```

When a NOT moves through a bracket, AND and OR swap, and every literal inside is complemented. This is needed when complementing grouped logic or converting a design into NAND/NOR-friendly form. It is different from factoring or distributing terms.

Why both forms are useful

A truth table may produce fewer gates in one form than the other. The exercise requests both so that you can simplify from the 1-cells and independently from the 0-cells, then recognize that the resulting circuits satisfy the same specified rows.

ENTRY 164 / ACCEPTED SOLUTION

Applying SOP and POS to the problem

The two accepted assignments are equivalent for every input, and they meet all of the problem's specified truth-table rows.

```
assign out_sop = (c & d) | (~a & ~b & c);
assign out_pos = c & (d | ~a) & (d | ~b);
```

Read the names from the outside inward

The SOP has two ANDed product terms joined by OR. The POS has the literal `c` and two ORed sum terms joined by AND. A single literal may act as a one-literal product or sum term.

Why these two lines are equivalent

```
(c & d) | (~a & ~b & c)
= c & (d | (~a & ~b))            // factor c
= c & (d | ~a) & (d | ~b)        // distribute OR over AND
```

That conversion uses the Boolean distributive identity `X | (Y & Z) = (X | Y) & (X | Z)`. It is **not** a De Morgan step because no NOT covers the bracket.

Required output	abcd row numbers	Meaning
1	2, 7, 15	The expression must be true.
0	0, 1, 4, 5, 6, 9, 10, 13, 14	The expression must be false.
Don't care	3, 8, 11, 12	Either value is allowed and may aid simplification.

The four don't-care rows can be chosen as 0 or 1 during minimization. Because of that freedom, more than one equally small POS may be valid; the accepted pair above deliberately implements one consistent value even on the don't-care rows.

Source: HDLBits Exams/ece241_2013_q2.

ENTRY 168 / RULE 90

One whole generation changes at a time

For every cell i, Rule 90 ignores the current center bit and XORs the current left and right neighbours.

```
next_q[i] = q[i-1] ^ q[i+1]
```

The eight next-state results, read from neighbourhood 111 down to 000, are `01011010`. Interpreted as a binary number, that is decimal 90 - hence the rule's name.

Left	Center	Right	Next center
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	1
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	0

Boundary cells

The imaginary neighbours outside the 512-cell vector are zero: `q[-1] = 0` and `q[512] = 0`. Therefore `next_q[0] = q[1]` and `next_q[511] = q[510]`.

The same rule as one vector expression

```
{q[510:0], 1'b0} ^ {1'b0, q[511:1]}
```

The first shifted vector places `q[i-1]` at bit i; the second places `q[i+1]` at bit i. The inserted zeros implement the two outside boundaries.

All cells must use the same old generation. They are not updated one by one in time. The design computes all 512 next bits as combinational logic, then the clock stores them together.

ENTRY 168 / YOUR ACCEPTED STRUCTURE

Why nextVal is used - and whether it is required

nextVal is a 512-bit combinational candidate for the next generation; q is the 512-bit register holding the current generation.

```
integer i;
reg [511:0] nextVal;

always @(posedge clk) begin
    if (load)
        q <= data;
    else
        q <= nextVal;
end

always @(*) begin
    nextVal[0]   = q[1];
    nextVal[511] = q[510];
    for (i = 1; i < 511; i = i + 1)
        nextVal[i] = q[i-1] ^ q[i+1];
end
```

Why you used it

The combinational block calculates every bit of the upcoming state from the unchanged current `q`. At the rising edge, the clocked block either loads `data` or captures that complete candidate with the nonblocking assignment `q <= nextVal`. This cleanly separates **current state** from **next-state logic**.

Is nextVal actually required?

No. It is required by this two-process organization, but Rule 90 does not require a separate named signal. The same next-state network can be written directly:

```
always @(posedge clk) begin
    if (load) q <= data;
    else      q <= {q[510:0], 1'b0} ^ {1'b0, q[511:1]};
end
```

Both styles describe essentially the same XOR wiring and 512 state bits. `nextVal` improves readability and is convenient in a waveform; it does not add another clock cycle or another bank of registers. The synthesis loop is unrolled into hardware rather than executing 510 times over time.

Two important safety points

Every bit of `nextVal` is assigned (two boundaries plus indices 1 through 510), so the combinational block does not infer a latch. Also avoid updating `q` in place with blocking assignments inside a loop: later iterations could read already-changed neighbours and would no longer represent a simultaneous generation.

Source: HDLBits Rule90. The current submission panel was rechecked and shows **Status: Success!**.

ENTRY 172 / RULE 110

Rule 110 uses left, center and right

Unlike Rule 90, Rule 110 is asymmetric: swapping the left and right neighbours can change the answer. Keep the vector direction fixed while translating the table.

Left	Center	Right	Next center
1	1	1	0
1	1	0	1
1	0	1	1
1	0	0	0
0	1	1	1
0	1	0	1
0	0	1	1
0	0	0	0

Read from neighbourhood 111 down to 000, the next-state column is `01101110`, which is decimal 110.

A compact Boolean form

```
next = (~L & C) | (C & ~R) | (~C & R)
     = (R ^ C) | (R & ~L)
```

The XOR term handles the rows where center and right differ. The extra `R & ~L` term adds neighbourhood 011, the remaining row whose output is 1.

Vector direction in your accepted code

```
L = q[i+1]     C = q[i]     R = q[i-1]
```

With that mapping, `(R ^ C) | (R & ~L)` becomes exactly the interior expression you submitted. This direction also explains the successful trace from a loaded value of 1: `1, 3, 7, d, 1f, 31, 73, d7, 1fd, 307`.

Direction is the important difference from Rule 90. Rule 90 XORs left and right, so mirroring them changes nothing. Rule 110 gives them different roles.

ENTRY 172 / YOUR ACCEPTED STRUCTURE

Why the Rule 110 expression works

The structure again separates the registered current generation q from the combinational next generation nextval.

```
reg [511:0] nextval;
integer i;

always @(posedge clk) begin
    if (load)
        q <= data;
    else
        q <= nextval;
end

always @(*) begin
    nextval[511] = q[511] | q[510];
    nextval[0]   = q[0];
    for (i = 1; i < 511; i = i + 1)
        nextval[i] = q[i-1] ^ q[i] | ~q[i+1] & q[i-1];
end
```

How Verilog groups the interior expression

```
nextval[i] = (q[i-1] ^ q[i])
           | ((~q[i+1]) & q[i-1]);
```

Unary ~ binds first, then &, then ^, and finally |. The added parentheses do not change the hardware; they make the intended `(R ^ C) | (R & ~L)` form obvious.

Why the two boundary assignments are different

At bit 0, the outside right neighbour is 0, so the rule reduces to `nextval[0] = q[0]`. At bit 511, the outside left neighbour is 0, so the rule reduces to `nextval[511] = q[511] | q[510]`. These are simplifications of the same rule, not special exceptions.

Why nextval is still useful

As in Rule 90, `nextval` lets all 512 cells be calculated from the unchanged current q before one clock edge stores the whole result. Every bit is assigned, so no latch is inferred. The name is convenient, not mandatory; a direct clocked implementation with nonblocking assignments could describe the same state update.

Source: HDLBits Rule110. The current compile-and-simulate panel was rechecked and shows **Status: Success!**.

SERIES MAP / ENTRIES 144, 149, 155, 160, 165, 170

One timer, six connected ideas

The counter is the timing primitive. The next five exercises progressively add a shift/down register, a 1101 recognizer, a four-cycle capture controller, the complete controller, and finally the datapath.

Entry	Problem	What it contributes
144	Counter with period 1000	A precise 0...999 timebase.
149	Shift register + down counter	One register with mutually exclusive shift/count modes.
155	1101 recognizer	Find the start word and remember that it was found.
160	Enable shifting for four cycles	Turn a one-cycle event into a four-edge capture window.
165	Complete FSM	Sequence search -> capture -> count -> acknowledge.
170	Complete timer	Combine controller and datapath with exact boundaries.

Questions preserved from the work

Why can a clocked block test next_state? Is next_state one clock late? When should the block test state instead? Is the synchronous-reset structure correct?

Why can I not write assign count = 0 as well as count <= ...? What exactly are multiple drivers, and why can different tools report them differently?

Do I need to clear the four-cycle counter? Why is the fourth-bit test 3 rather than 4? How does (delay + 1) x 1000 arise without multiplication, and what does count represent?

Entry 155 appears in an earlier note too. It is repeated here deliberately because its transition into the capture phase is the timing hinge for the whole series.

ENTRY 144 / PERIOD-1000 COUNTER

Count 0 through 999, then wrap

A 10-bit register can represent 0 through 1023, so width alone does not create a modulo-1000 counter. The terminal-value comparison supplies the missing wrap rule.

```
always @(posedge clk) begin
    if (reset)
        q <= 10'd0;
    else if (q == 10'd999)
        q <= 10'd0;
    else
        q <= q + 10'd1;
end
```

Why compare with 999, not 1000?

The value visible during a cycle is the value stored after the preceding rising edge. When that stored value is 999, the next edge must write 0. Testing 1000 would expose an unwanted 1000 cycle and create a period of 1001.

Before edge	reset	Value written at edge	Visible after edge
998	0	998 + 1	999
999	0	0 (explicit wrap)	0
anything	1	0 (reset wins)	0

Synchronous reset and register ownership

Synchronous means reset is sampled only on a rising edge. Between edges, changing reset does not immediately change q. Declaring q as output reg lets the clocked process own it directly. Using an internal reg count plus assign q = count is equivalent; that continuous assignment drives q, not count.

The reset branch does not need a separate count=0 declaration elsewhere. One clocked process gives the stored value one owner, and every non-reset edge either wraps or increments it.

Source: HDLBits Exams/review2015_count1k. The open submission shows Status: Success!

ENTRY 149 / SHIFT + COUNT DATAPATH

MSB-first input still shifts toward the MSB

The first serial bit must eventually become q[3]. Appending each new bit at q[0] moves every older bit one place toward q[3].

```
always @(posedge clk) begin
    if (shift_ena)
        q <= {q[2:0], data};
    else if (count_ena)
        q <= q - 4'd1;
end
```

Capture edge	data	q after edge	Meaning
1	1	0001	First bit is newest at q[0].
2	0	0010	First bit moves to q[1].
3	0	0100	First bit moves to q[2].
4	1	1001	Serial 1001 is now q[3:0].

Why the rightmost item is data

Concatenation writes q[3:1] from the old q[2:0] and writes q[0] from the current serial input. Calling the stream MSB-first describes the order in which the four-bit number arrives; it does not mean each new bit is inserted directly into q[3].

Why q <= q - 1 wraps from 0 to 15

The destination is four bits. The mathematical result -1 is represented modulo 16, so its low four bits are 1111. An explicit 0-to-15 branch is correct and readable, but it is not required for a four-bit decrement.

There is no reset in this component's interface. Simulation may therefore show X until an enabled edge establishes a known value. The final system guarantees four shift edges before it counts, so that behavior is intentional.

Source: HDLBits Exams/review2015_shiftcount. The open submission shows Status: Success!

ENTRY 149 / PRIORITY AND OLD VALUES

Two true enables: the last assignment wins

The exercise promises that shift_ena and count_ena are not used together. Your two-if version is therefore accepted, but it still has a definite simulation meaning.

```
if (shift_ena)
    q <= {q[2:0], data};
else
    q <= q;                 // unnecessary self-assignment

if (count_ena)
    q <= q - 4'd1;          // later assignment in same process
```

All right-hand sides use the old q

Nonblocking assignments sample their right-hand sides during the active clock event and update the destination later. If both enables are 1, both calculations use the same pre-edge q, and the later scheduled write wins. The result is old q minus one; it is not the shifted value minus one.

shift_ena	count_ena	Two-if result	Explicit if/else-if result
0	0	hold	hold
1	0	shift	shift
0	1	decrement	decrement
1	1	decrement (last NBA)	shift (declared priority)

Three cleanup rules

1. Omit q <= q in a clocked process: no assignment already means hold. 2. Use if/else if when you want a visible priority. 3. Give a stored signal one procedural owner; assignment order is predictable only within that one process.

Why this is different from multiple drivers

Several nonblocking assignments to q inside one clocked block are competing assignments from one owner, resolved by source order. Assigning q from another always block or continuous assign creates another owner. That is the multiple-driver problem addressed on page 17.

ENTRY 155 / 1101 RECOGNIZER

The repeated question that unlocks the series

Five states record the longest useful suffix seen so far. The final state is sticky in this component because later exercises replace that self-loop with the four-bit capture phase.

State	Remembered suffix	Next on 0	Next on 1
SEARCH0	none	SEARCH0	SEARCH1
SEARCH1	1	SEARCH0	SEARCH11
SEARCH11	11	SEARCH110	SEARCH11
SEARCH110	110	SEARCH0	FOUND
FOUND	1101 found	FOUND	FOUND

> “Why next_state? next_state is a clock late ... is reset right?”

next_state is not another stored register here. It is combinational logic calculated from the current state and current data. On the edge that samples the final 1, next_state is FOUND before the nonblocking updates; state becomes FOUND after that edge.

```
always @(posedge clk) begin
    if (reset) begin
        state          <= SEARCH0;
        start_shifting <= 1'b0;
    end else begin
        state <= next_state;
        if (next_state == FOUND)
            start_shifting <= 1'b1;  // intentional sticky register
    end
end
```

This raises the registered output on the same edge that state enters FOUND. Testing state == FOUND in that clocked block would raise it one edge later. The cleaner Moore form is assign start_shifting = (state == FOUND): after the state update, the decode rises in the FOUND cycle without a separate output register.

The synchronous reset is correct: when reset is 1 at a rising edge, its branch wins and the next_state calculation is ignored for that state update.

Source: HDLBits Exams/review2015_fsmseq. The open submission shows Status: Success!

ENTRY 160 / FOUR-CYCLE ENABLE

Exactly four cycles, with one owner per register

The accepted counter starts at one on the reset edge while shift_ena becomes 1. Three more enabled edges advance the stored count to four; the following cycle is disabled.

Edge	Cause	count after edge	shift_ena after edge
R	reset=1	1	1
R+1	count < 4	2	1
R+2	count < 4	3	1
R+3	count < 4	4	1
R+4	count is 4	holds 4	0

> “Why can’t I add assign count = 3'd0? What does multiple drivers mean, and does it behave differently in each simulator?”

A continuous assign drives its left side all the time. The clocked block also tries to drive count after clock edges. Those are two hardware sources connected to one signal. In Verilog, a procedural destination is a variable/reg while a continuous-assignment destination is a net; mixing the two is normally illegal. If a resolved net does have two drivers, conflicting 0 and 1 values resolve to X in simulation and do not describe an ordinary FPGA register.

Pattern	Meaning	Use it?
count <= ... in one clocked block	One flip-flop bank owns count.	Yes
assign q = count	Internal count drives a separate output net q.	Yes
assign count = 0 plus count <= ...	Constant driver fights the clocked driver.	No
count <= ... in two always blocks	Two procedural owners; ordering is not a priority rule.	No

Tool messages vary—illegal reg drive, multiple drivers, unresolved signal, or synthesis failure—but the design rule does not: one signal, one owner. Initialization belongs in the reset branch of that owner.

Source: HDLBits Exams/review2015_fsmshift. The open submission shows Status: Success!

ENTRY 165 / COMPLETE CONTROLLER

Search, capture, count, notify, acknowledge

The controller combines the previous state machines but still treats the counters as an external datapath. Its outputs describe phases; done_counting returns the datapath result.

Phase	State meaning	Output/action	Exit condition
Search	Remember suffixes of 1101	All controls 0	Final 1 arrives
Capture	Receive four following bits	shift_ena=1	Four capture cycles complete
Count	Wait for external counters	counting=1	done_counting=1
Notify	Timeout is visible	done=1	ack=1

A state-per-cycle controller

```
SEARCH110: next_state = data ? BIT0 : SEARCH0;
BIT0:      next_state = BIT1;
BIT1:      next_state = BIT2;
BIT2:      next_state = BIT3;
BIT3:      next_state = COUNTING;
COUNTING:  next_state = done_counting ? WAIT_ACK : COUNTING;
WAIT_ACK:  next_state = ack ? SEARCH0 : WAIT_ACK;

assign shift_ena = (state == BIT0) || (state == BIT1) ||
                   (state == BIT2) || (state == BIT3);
assign counting  = (state == COUNTING);
assign done      = (state == WAIT_ACK);
```

This version needs more states but no separate four-cycle counter. A compressed CAPTURE state plus a two- or three-bit counter is equally valid. State count and datapath count are two representations of the same four-cycle history.

Why Moore-style output decoding is useful

Outputs are determined from the current phase, so there is one obvious cycle definition: shift_ena is high for every cycle spent in BIT0 through BIT3; counting is high while waiting; done stays high until acknowledgement. No extra output register can drift away from the state.

Source: HDLBits Exams/review2015_fsm. The open result panel shows Status: Success!

ENTRY 165 / COUNTER-COMPRESSED VERSION

Yes, the capture counter must be re-armed

Your accepted controller uses one CAPTURE state and count to remember which of the four capture cycles is active. That register must be ready at zero before every new timer transaction.

> “Do I need to make count 0 or not?”

For a controller that can return from WAIT_ACK to searching and start again, yes. If count remains 4, the next arrival in CAPTURE immediately appears complete and shift_ena never supplies four fresh cycles. Resetting count outside CAPTURE, resetting it when CAPTURE finishes, or setting it on entry are all valid—choose one clear owner and one convention.

```
always @(posedge clk) begin
    if (reset) begin
        state <= SEARCH0;
        count <= 0;
    end else begin
        state <= next_state;
        if (state == CAPTURE) begin
            if (count == 3)
                count <= 0;        // fourth cycle completed
            else
                count <= count + 1'b1;
        end else begin
            count <= 0;            // armed before next entry
        end
    end
end
assign shift_ena = (state == CAPTURE);
```

Why the terminal value is 3

With zero-based numbering, the four capture cycles are 0, 1, 2, and 3. During the cycle with count=3, shift_ena is still high and the fourth bit is sampled at its ending edge. That same edge may move state to COUNTING. Waiting for count=4 would create a fifth CAPTURE cycle.

CAPTURE cycle	count before edge	Bit sampled	state after edge
1	0	delay[3]	CAPTURE
2	1	delay[2]	CAPTURE
3	2	delay[1]	CAPTURE
4	3	delay[0]	COUNTING

CLOCKED LOGIC / THE CENTRAL CONFUSION

state is now; next_state is the chosen destination

Both names are evaluated before the nonblocking update at a rising edge. They answer different questions, so neither is universally the correct test.

Before the edge	At the edge	After NBA updates
state stores Q input has D next_state = F(Q,D)	Clocked blocks read Q, D, and F(Q,D). Every <= RHS is sampled.	state stores F(Q,D). Registered outputs receive their scheduled values.

```
always @(posedge clk) begin
    state <= next_state;

    if (state == CAPTURE)
        delay <= {delay[2:0], data};   // action of current cycle

    if (next_state == FOUND)
        found_reg <= 1'b1;             // react to chosen transition
end
```

Testing state

Use state when the action belongs to the phase the machine is already in: sample a delay bit during CAPTURE, advance a subcounter during COUNTING, or clear phase-local storage while outside that phase. This reads the pre-edge current state.

Testing next_state

Use next_state only when you intentionally want a registered reaction to the transition decision made from this edge's inputs. In the standalone recognizer, next_state==FOUND sets a sticky output on the edge that consumes the final 1. It is not a universal shortcut for avoiding a delay.

A decoded Moore output needs neither test in the sequential block: assign done = (state == WAIT_ACK). The state update itself makes the output change in the new state.

next_state is usually combinational—not a second clocked register and not inherently one cycle late.

CLOCKED LOGIC / DECISION GUIDE

When to use state, next_state, or both

Start by naming the event you mean. Then choose the expression that identifies that event exactly.

Intent	Recommended condition	Reason
Do work throughout phase X	state == X	X is the current cycle.
Decode a Moore output	assign out = (state == X)	Keeps output aligned with state.
React on transition into X	state != X && next_state == X	A true one-edge entry event.
React on transition out of X	state == X && next_state != X	A true one-edge exit event.
Register an output high whenever destination is X	next_state == X	Includes entry and any X self-loop.
Preserve a sticky event	set on entry; omit assignment otherwise	Clocked omission intentionally holds.

Why next_state == X is not an entry detector

If X self-loops, next_state remains X every cycle inside X. The condition is therefore true on entry and during the stay. Use state != X && next_state == X when exactly one entering edge matters.

Why delay capture must use state == CAPTURE

On the edge that completes 1101, current state is SEARCH110 and next_state is CAPTURE. Testing next_state would shift that final recognizer bit into delay. Four such tests would capture 1000 for an intended following delay 0001. Testing state starts on the next edge, when the first actual delay bit is present.

Reset and ownership checklist

In a synchronous-reset block, reset wins only at a rising edge. Reset state and every independently stored datapath register that must begin known. next_state can still toggle combinationally while reset is high; the reset branch ignores it. Assign next_state in the combinational block only, and state in the clocked block only.

Do not assign next_state from both blocks. Do not register next_state unless you deliberately want another pipeline stage.

ENTRY 170 / CAPTURE THE DELAY

The fourth bit uses old delay plus current data

Nonblocking assignment means delay still contains the first three captured bits while the fourth edge is being evaluated. The concatenation supplies the missing current bit immediately.

```
if (state == CAPTURE) begin
    delay <= {delay[2:0], data};
    if (bit_index == 3) begin
        bit_index <= 0;
        remaining <= {delay[2:0], data};
        subcount <= 0;
    end else begin
        bit_index <= bit_index + 1'b1;
    end
end
```

Before edge	Current data	Candidate {delay[2:0],data}	After edge
delay=0000, index=0	0	0000	delay=0000
delay=0000, index=1	0	0000	delay=0000
delay=0000, index=2	0	0000	delay=0000
delay=0000, index=3	1	0001	delay=0001; remaining=0001

> “At the third tick, if I put the completed value in countreg, do I still need next_state?”

The bit index and the state answer separate questions. bit_index==3 says this edge has the fourth delay bit, so it constructs the completed value. state==CAPTURE says the current data belongs to the delay field at all. next_state is not needed for the datapath write; combinational FSM logic independently uses bit_index==3 to leave CAPTURE on that edge.

Why the if test is == 3, not != 3

The special work—copying the completed four-bit candidate into remaining—belongs only to the fourth capture edge. Using != 3 would perform it on the first three edges and skip it precisely when the final bit arrives.

Source: HDLBits Exams/review2015_fancytimer. The open result panel shows Status: Success!

ENTRY 170 / EXACT TIMER LENGTH

Two nested counters implement (delay + 1) x 1000

remaining is not the raw clock-cycle count. It is the value displayed for a block of 1000 cycles. subcount selects the cycle within that block.

```
if (state == COUNTING) begin
    if (subcount == 10'd999) begin
        subcount <= 0;
        if (remaining != 0)
            remaining <= remaining - 1'b1;
    end else begin
        subcount <= subcount + 1'b1;
    end
end

// Leave after the final cycle of the final block.
COUNTING: next_state = (remaining == 0 && subcount == 999)
                       ? WAIT_ACK : COUNTING;
```

Loaded delay	remaining shown	Cycles per value	Total COUNTING cycles
0	0	1000	1000
1	1, then 0	1000 each	2000
5	5,4,3,2,1,0	1000 each	6000
15	15 down to 0	1000 each	16000

> “How is this block making (delay + 1) x 1000? What is count? Am I putting delay in countreg?”

Yes: remaining/countreg starts as the captured delay. Values delay, delay-1, ..., 0 form delay+1 display blocks. subcount visits 0 through 999 inside every block, giving 1000 cycles per value. Multiplication is unnecessary because the two counters express the product structurally.

The terminal comparison is made from the old pre-edge values. When remaining=0 and subcount=999, that cycle is still the thousandth cycle displaying 0; the ending edge moves to WAIT_ACK. This avoids the two common off-by-one failures: leaving at 998 or creating an extra cycle at 1000.

A 10-bit subcount is sufficient because 999 fits. A nine-bit register is not: its maximum is 511. A separate product register would need 14 bits for the maximum 16000, but this architecture never stores that product.

ENTRY 170 / REFERENCE RTL, PART 1

Controller and next-state logic

This complete version keeps the successful seven-state structure, uses one owner per stored signal, and makes every next-state path explicit.

```
module top_module (
    input clk,
    input reset,          // Synchronous reset
    input data,
    output [3:0] count,
    output counting,
    output done,
    input ack
);
    localparam SEARCH0   = 3'd0,
               SEARCH1   = 3'd1,
               SEARCH11  = 3'd2,
               SEARCH110 = 3'd3,
               CAPTURE   = 3'd4,
               COUNTING  = 3'd5,
               WAIT_ACK  = 3'd6;

    reg [2:0] state, next_state;
    reg [2:0] bit_index;
    reg [3:0] delay_shift;
    reg [3:0] remaining;
    reg [9:0] subcount;

    always @(*) begin
        case (state)
            SEARCH0:   next_state = data ? SEARCH1 : SEARCH0;
            SEARCH1:   next_state = data ? SEARCH11 : SEARCH0;
            SEARCH11:  next_state = data ? SEARCH11 : SEARCH110;
            SEARCH110: next_state = data ? CAPTURE : SEARCH0;
            CAPTURE:   next_state = (bit_index == 3) ? COUNTING : CAPTURE;
            COUNTING:  next_state = (remaining == 0 && subcount == 999)
                                    ? WAIT_ACK : COUNTING;
            WAIT_ACK:  next_state = ack ? SEARCH0 : WAIT_ACK;
            default:   next_state = SEARCH0;
        endcase
    end
```

Why the default branch matters

Every legal state assigns next_state, and default recovers from an unused or unknown encoding. This is complete combinational logic: it does not intentionally remember an earlier next_state value and therefore does not infer a latch.

The SEARCH11 self-loop preserves overlap. After 111, the most useful suffix is still 11, so a following 01 can complete the pattern.

ENTRY 170 / REFERENCE RTL, PART 2

Datapath, outputs and a final audit

The sequential block performs work for the current phase; continuous Moore decodes expose the phase after each state update.

```
    always @(posedge clk) begin
        if (reset) begin
            state       <= SEARCH0;
            bit_index   <= 0;
            delay_shift <= 0;
            remaining   <= 0;
            subcount    <= 0;
        end else begin
            state <= next_state;

            if (state == CAPTURE) begin
                delay_shift <= {delay_shift[2:0], data};
                if (bit_index == 3) begin
                    remaining <= {delay_shift[2:0], data};
                    bit_index <= 0;
                    subcount  <= 0;
                end else begin
                    bit_index <= bit_index + 1'b1;
                end
            end else begin
                bit_index <= 0;
            end

            if (state == COUNTING) begin
                if (subcount == 10'd999) begin
                    subcount <= 0;
                    if (remaining != 0)
                        remaining <= remaining - 1'b1;
                end else begin
                    subcount <= subcount + 1'b1;
                end
            end
        end
    end

    assign count    = remaining;
    assign counting = (state == COUNTING);
    assign done     = (state == WAIT_ACK);
endmodule
```

Final audit

Check	What to confirm
Sampling	The final 1 of 1101 is not the first delay bit.
Capture	Exactly four edges shift MSB-first; index 3 completes the value.
Timing	subcount spans 0...999 for every remaining value, including 0.
Outputs	counting and done decode current state; done holds until ack.
Reset	State and all independently stored datapath values reset on the edge.
Ownership	Each register is written by one clocked block; next_state by one combinational block.

Verification used for this note

The builder checks the 998,999,0,1 wrap; MSB-first capture of 1001; four-bit underflow; entry to CAPTURE after 1101; the wrong 1000 value produced by consuming the recognizer's final 1; and every delay value 0 through 15 for exact duration and 1000-cycle count plateaus.

The six open HDLBits pages showed successful submissions. That platform evidence establishes acceptance; the local model above independently checks the timing claims printed in this note.

SERIES MAP / ENTRIES 141, 145, 151

Three LFSRs, one translation method

These exercises move from a drawn five-bit Galois LFSR, through a three-bit mux-and-flip-flop circuit, to a 32-bit Galois LFSR. The reliable workflow is always the same: name the old state, write each next-bit equation, and only then compress the equations into Verilog.

Entry	Problem	Main reasoning task
141	Lfsr5	Translate one-based taps 5 and 3 into a right-shifting Galois update.
145	Mt2015_lfsr	Read the mux diagram, distinguish parallel load from feedback, and map board pins.
151	Lfsr32	Scale the same Galois pattern to taps 32, 22, 2, and 1 without an index error.

Questions this note answers

What does an LFSR actually store, and why can a small register generate a long sequence without using a counter?

If the diagram says tap position 3, why does the code override q[2]? Why does tap 22 become q[21] in the 32-bit version?

Why is a whole-vector nonblocking assignment followed by a few slice assignments valid? Do the XOR expressions see old q or the partly shifted q?

For Mt2015_lfsr: what exactly is the question asking, how do L, R, Clock, Q, SW, KEY, and LEDR correspond, and why does the accepted solution work?

All three open HDLBits result panels showed Status: Success! on 9 September 2026. The note also verifies the full 31-state and 7-state cycles locally.

SHARED MODEL / STATE RECURRENCE

An LFSR is a state machine written as bit equations

An n-bit register stores one state. At each active clock edge, every next bit is a linear XOR expression of the old bits. Repeating that fixed transformation produces a deterministic orbit through the state space.

The three layers

Layer	Question to ask	Example
State	What values exist before this edge?	old q[4:0] = 00001
Next-state equations	What should each destination bit become?	next q[2] = old q[3] ^ old q[0]
Clocked storage	When do all those values become visible?	Together, immediately after posedge clk

Why XOR is called linear

Over one-bit arithmetic, XOR is addition modulo 2: 0^0=0, 0^1=1, 1^0=1, and 1^1=0. Every LFSR next-state bit is a sum of selected old bits with no carries. That is why matrix and polynomial methods can analyze its period.

Maximum length and the missing state

An n-bit register has 2^n possible bit patterns. A maximum-length XOR-feedback LFSR visits all 2^n - 1 nonzero states before repeating. It cannot include zero because XORs of all zeros are still zero; 000...0 is a separate self-loop called lockup.

Resetting to 1 is therefore functional, not decorative. It chooses a nonzero seed. Any other nonzero seed on the same maximal cycle changes the starting point but not the cycle length.

LFSR is not the same as binary count

The output sequence looks irregular and is useful for test patterns, scrambling, CRC-style logic, and compact counters. It is still fully deterministic. Knowing the current state determines every future state.

Recall test

If q is all zero and the only feedback operators are XOR, no clock edge can escape zero.

SHARED MODEL / TAP NUMBERING

Tap position k maps to Verilog index k-1

HDLBits numbers tap positions from 1, but Verilog vectors use zero-based indices. Write the position and index on separate lines before coding; this single step prevents the most common LFSR error.

Named position	Verilog bit	Five-bit meaning	Thirty-two-bit meaning
1	q[0]	Output/feedback bit	Output plus tap stage
2	q[1]	Ordinary shift stage	Tap stage
3	q[2]	Tap stage	Ordinary shift stage
22	q[21]	Not present	Tap stage
32	q[31]	Not present	Feedback enters here

Start from the ordinary right shift

```
next_q = {old_q[0], old_q[N-1:1]};
// next_q[N-1] = old_q[0]
// next_q[i]   = old_q[i+1] for 0 <= i < N-1
```

Then override the tap destinations

For a Galois tap at position k below the top position, the destination q[k-1] receives its ordinary shifted source XORed with the outgoing feedback bit q[0]. Thus position 3 gives next q[2] = old q[3] ^ old q[0]. Position 22 gives next q[21] = old q[22] ^ old q[0].

Why the top tap may not need a separate line

The base concatenation already writes old q[0] into the top stage. The diagram often draws the top XOR with a constant-zero second input for consistency. XOR with zero changes nothing, so the whole-vector shift already implements it.

Mechanical translation

Diagram position -> subtract 1 -> destination index. Ordinary shifted source -> add 1 -> old source index. XOR the outgoing old q[0] only at tapped destinations.

ENTRY 141 / LFSR5

The five-bit Galois solution

The circuit shifts right. Position 5 receives the outgoing q[0], while the tap at position 3 receives old q[3] XOR old q[0]. The synchronous reset selects seed 00001.

```
module top_module (
    input clk,
    input reset,              // Active-high synchronous reset
    output reg [4:0] q
);
    always @(posedge clk) begin
        if (reset) begin
            q <= 5'h01;
        end else begin
            q    <= {q[0], q[4:1]};
            q[2] <= q[3] ^ q[0];
        end
    end
endmodule
```

Line-by-line meaning

Code	Next-state equation
q <= {q[0], q[4:1]};	q4'=q0, q3'=q4, q2'=q3, q1'=q2, q0'=q1
q[2] <= q[3] ^ q[0];	Replace the base q2'=q3 with tapped q2'=q3^q0
q <= 5'h01;	Select nonzero seed 00001 when reset is high at the edge

Why q[2] is assigned twice

Both assignments are in the same always block. The whole-vector line supplies the default next value for every bit, then the later slice assignment replaces only q[2]. This is a compact default-plus-exception pattern, not two hardware drivers.

Why output reg is shown

The always block stores q, so q must be a procedural variable in Verilog. Declaring output reg [4:0] q makes the port itself the register. An internal reg plus assign q = internal_q would also be valid but adds an unnecessary name here.

Source: HDLBits Lfsr5. The page specifies taps 5 and 3, reset to 1, and shows Status: Success!

ENTRY 141 / LFSR5 TRACE

Old values produce the 31-state cycle

Tracing from reset seed 01 hex exposes both the shift direction and the tap override. It also matches the beginning of HDLBits' displayed reference waveform.

Step	old q	old q0	base shift	new q2	next q
0	00001 (01)	1	10000	q3^q0 = 1	10100 (14)
1	10100 (14)	0	01010	q3^q0 = 0	01010 (0A)
2	01010 (0A)	0	00101	q3^q0 = 1	00101 (05)
3	00101 (05)	1	10010	q3^q0 = 1	10110 (16)
4	10110 (16)	0	01011	q3^q0 = 0	01011 (0B)

The important first edge

Starting at 00001, an ordinary rotate-like right shift would produce 10000. The position-3 tap changes q[2] from old q[3]=0 to 0^1=1, so the real result is 10100. If your first post-reset state is not 10100, the shift direction or tap index is wrong.

A compact sequence fingerprint

```
01 -> 14 -> 0A -> 05 -> 16 -> 0B -> 11 -> 1C -> 0E -> ...
```

Local exhaustive check

The builder starts at 00001, records states until the first repeat, and asserts 31 unique states, no zero state, and a return to 00001. That verifies the implemented recurrence, not merely the first few waveform labels.

Reset timing

Because reset is inside always @(posedge clk), it is synchronous. A high reset between edges does not immediately change q. At the next rising edge, reset wins over the feedback update and q becomes 00001.

Debug boundary

Check one edge after reset first. A correct 00001 -> 10100 transition proves the seed, shift direction, feedback bit, and tap destination together.

ENTRY 145 / MT2015_LFSR

Read the schematic as three D-input equations

Each flip-flop stores one LEDR bit. Its input mux chooses either the parallel R input or the feedback-network value. The load selector L is shared, so all three bits load or advance together.

Schematic label	Top-level port	Role
R[2:0]	SW[2:0]	Three-bit value loaded in parallel
L	KEY[1]	Mux selection: 1 loads R, 0 selects feedback
Clock	KEY[0]	Rising edge stores all three selected D inputs
Q[2:0]	LEDR[2:0]	Visible stored state

Feedback equations when L=0

```
next Q[0] = old Q[2]
next Q[1] = old Q[0]
next Q[2] = old Q[1] ^ old Q[2]
```

Parallel-load equations when L=1

```
next Q[0] = R[0]
next Q[1] = R[1]
next Q[2] = R[2]
```

What the question is asking

The task is not asking you to instantiate physical switches, keys, LEDs, muxes, or flip-flop submodules. It asks for behavior equivalent to the shown circuit while using the required board-facing port names. One clocked always block implements all three flip-flops and their shared load selection.

Do not infer an extra reset

This module has no reset port. KEY[1] and SW supply an explicit load path instead. To establish a known state, set SW to a chosen seed, use L=1 for one active clock edge, then use L=0 to advance the recurrence.

Source: HDLBits Mt2015_lfsr. The page maps R to SW, Clock to KEY[0], L to KEY[1], and Q to LEDR.

ENTRY 145 / MT2015_LFSR RTL

Load has explicit priority over feedback

The if branch models the shared mux select. At every rising edge of KEY[0], KEY[1] chooses one complete three-bit next state: SW for load, or the feedback equations for advance.

```
module top_module (
    input  [2:0] SW,          // Parallel-load value R
    input  [1:0] KEY,         // KEY[1] = L, KEY[0] = clock
    output reg [2:0] LEDR     // Register outputs Q
);
    always @(posedge KEY[0]) begin
        if (KEY[1]) begin
            LEDR <= SW;
        end else begin
            LEDR[0] <= LEDR[2];
            LEDR[1] <= LEDR[0];
            LEDR[2] <= LEDR[1] ^ LEDR[2];
        end
    end
endmodule
```

Why LEDR <= SW is the right load

The vector assignment is exactly the three parallel equations LEDR[i] <= SW[i]. No shifting happens in that branch. Since it is the if branch, it has priority whenever KEY[1] is 1 at the sampling edge.

Why all three feedback assignments read old LEDR

Nonblocking <= evaluates all three right-hand sides from the same old LEDR. The first line does not alter the value read by the second, and neither alters the operands read by the XOR. They represent three flip-flops sampling simultaneously.

At posedge KEY[0]	KEY[1]	Stored result
Load operation	1	LEDR_next = SW
Sequence operation	0	LEDR_next = {LEDR1^LEDR2, LEDR0, LEDR2}

Board polarity versus logic polarity

The exercise connects KEY[1] directly to logical L and the accepted circuit loads when that signal is 1. A physical pushbutton may be electrically active-low on the board; that affects how a person drives the pin, not the Boolean behavior requested by the schematic. Do not silently insert an inversion that the problem does not show.

Original saved question

The editor comment asked to understand the question before the solution. The mapping table and the two equation sets on page 31 provide that pre-code interpretation; this page is the direct RTL transcription.

ENTRY 145 / MT2015_LFSR TRACE

Seven nonzero states prove the recurrence

Load seed 001, deassert L, and advance once per rising edge. The circuit visits all seven nonzero three-bit patterns before returning to 001.

Old Q	Q0'=old Q2	Q1'=old Q0	Q2'=old Q1^old Q2	Next Q
001	0	1	0^0=0	010
010	0	0	1^0=1	100
100	1	0	0^1=1	101
101	1	1	0^1=1	111
111	1	1	1^1=0	011
011	0	1	1^0=1	110
110	1	0	1^1=0	001

Why 000 never appears

Every listed nonzero state maps to another nonzero state. Separately, 000 maps to 000 because each next bit is either an old bit or XOR of old bits. Loading 000 therefore locks the generator. Load any of the seven nonzero seeds to enter the same seven-state cycle at a different point.

What would blocking assignments risk?

If the feedback branch used = and the statements executed sequentially, later equations could read values already changed by earlier statements. The simulated recurrence would depend on source order and no longer match three simultaneous D flip-flops. Nonblocking <= preserves the intended old-state snapshot.

A useful mental rewrite

```
old = LEDR;
LEDR_next[0] = old[2];
LEDR_next[1] = old[0];
LEDR_next[2] = old[1] ^ old[2];
```

You do not need this temporary variable in the final RTL. It is a reasoning device that makes simultaneous sampling explicit.

Verification result

001 -> 010 -> 100 -> 101 -> 111 -> 011 -> 110 -> 001. Period = 7 = 2^3 - 1.

ENTRY 151 / LFSR32

The 32-bit solution is the five-bit pattern scaled up

Start with the same right-shift default, then override destinations for the lower taps. Positions 22, 2, and 1 become q[21], q[1], and q[0]. Position 32 is already handled by feedback into q[31].

```
module top_module (
    input clk,
    input reset,              // Active-high synchronous reset
    output reg [31:0] q
);
    always @(posedge clk) begin
        if (reset) begin
            q <= 32'h00000001;
        end else begin
            q     <= {q[0], q[31:1]};
            q[21] <= q[22] ^ q[0];
            q[1]  <= q[2]  ^ q[0];
            q[0]  <= q[1]  ^ q[0];
        end
    end
endmodule
```

Tap position	Destination index	Ordinary source	Implemented next value
32	q[31]	constant 0 at drawn XOR	old q[0]
22	q[21]	old q[22]	old q[22] ^ old q[0]
2	q[1]	old q[2]	old q[2] ^ old q[0]
1	q[0]	old q[1]	old q[1] ^ old q[0]

The off-by-one trap

Writing q[22], q[2], and q[1] treats the one-based tap labels as if they were Verilog indices. That moves every lower XOR one stage too high. The first visible state can expose the error immediately.

First edge after reset seed 00000001

```
base shift: q[31]=1, all other bits=0
tap overrides: q[21]=1, q[1]=1, q[0]=1
next q = 32'h80200003
```

Source: HDLBits Lfsr32. The page specifies taps 32, 22, 2, and 1 and reset to 32'h1.

ENTRY 151 / NONBLOCKING OVERRIDES

The XORs read old q, never a partly shifted q

The accepted compact style relies on two Verilog rules: nonblocking right-hand sides are sampled before updates become visible, and the last assignment to an overlapping destination in one process determines that destination's scheduled value.

Statement	RHS snapshot	Scheduled destination
q <= {q[0],q[31:1]}	Entire old q	All 32 next bits
q[21] <= q[22]^q[0]	old q[22], old q[0]	Replace scheduled bit 21
q[1] <= q[2]^q[0]	old q[2], old q[0]	Replace scheduled bit 1
q[0] <= q[1]^q[0]	old q[1], old q[0]	Replace scheduled bit 0

This is one owner, not multiple drivers

All four assignments live in one clocked process. Synthesis sees one bank of 32 flip-flops with combinational next-value logic. A separate always block or continuous assign writing q would create another owner and would be a real multiple-driver error.

Equivalent explicit next-vector style

```
reg [31:0] next_q;
always @(*) begin
    next_q     = {q[0], q[31:1]};
    next_q[21] = q[22] ^ q[0];
    next_q[1]  = q[2]  ^ q[0];
    next_q[0]  = q[1]  ^ q[0];
end
always @(posedge clk)
    if (reset) q <= 32'h1; else q <= next_q;
```

Both styles describe the same hardware. The explicit form can be easier to audit because every exception names next_q; the compact form is shorter and safe when all overlapping nonblocking assignments remain in one process.

Polynomial check

The stated taps correspond to x^32 + x^22 + x^2 + x + 1, up to the reciprocal orientation used by the right-shifting implementation. The builder verifies the primitive-order conditions using the prime factors 3, 5, 17, 257, and 65537 of 2^32 - 1. This supports the maximal nonzero period stated by the exercise.

REFERENCE / VERIFICATION AND DEBUGGING

Use equations first, code second

The shortest reliable LFSR solution starts from a paper trace. Treat every HDLBits drawing as a next-state specification, then use one clocked owner to store the result.

Check	What to write or test
1. Direction	For each destination, identify the old source before thinking about taps.
2. Numbering	Convert one-based position k to zero-based q[k-1].
3. Feedback	Mark the outgoing old bit once; use it only at drawn tap stages.
4. Storage	Use <= in one edge-triggered owner. Omitted assignment means hold.
5. Seed	Avoid zero for XOR-feedback operation; match the stated reset or load.
6. First edge	Predict one post-seed value by hand and compare it exactly.
7. Period	For small designs, enumerate until repeat and assert zero is absent.

Three reference results

Problem	Seed or load	First result / verified cycle
Lfsr5	00001	10100; all 31 nonzero states, then 00001
Mt2015_lfsr	001 via SW	010; seven-state cycle ending 110 -> 001
Lfsr32	00000001	80200003; primitive-polynomial order check passes

Mistake-to-symptom map

Symptom	Likely cause
First Lfsr5 result is 10000	The tap-3 override was omitted.
Lfsr32 lower XORs appear one stage high	Tap positions were used directly as indices.
Mt2015 result changes with statement order	Blocking = was used for clocked state updates.
Sequence stays zero	Zero was reset or loaded into an XOR-feedback LFSR.
Compiler rejects q as an l-value	The procedurally assigned output was not declared reg/variable.

Verification used for this note

The builder exhaustively verifies the 5-bit period of 31 and the 3-bit period of 7, checks their exact early sequences, checks the 32-bit first transition 00000001 -> 80200003, and proves the stated 32-bit polynomial has full multiplicative order. The three current HDLBits panels independently show accepted submissions.

ENTRY 177 / GRID REPRESENTATION

Why the index is row * 16 + col

The circuit stores 256 cells in q[255:0]. A row has 16 cells, so the start of row r is bit 16r. Column c selects the c-th bit after that start.

Your editor question: "why and how to calc the row and col index, discuss."

Derive the address from rows already skipped

Before row 3 there are three complete rows: 3 x 16 = 48 cells. Moving to column 5 adds five more positions. Therefore cell (3,5) is q[53]. Both coordinates start at zero. Multiplication by 16 supplies the row offset; addition supplies the position inside that row.

Coordinate or row	Vector location	Reason
(0,0)	q[0]	0 x 16 + 0
Row 0	q[15:0]	Indices 0 through 15
Row 1	q[31:16]	Indices 16 through 31
(3,5)	q[53]	48 + 5
(15,15)	q[255]	240 + 15

Recover the coordinates

```
index = row * 16 + col;
row   = index / 16;  // integer quotient
col   = index % 16;  // remainder
```

For index 53, integer division gives row 3 and the remainder gives column 5. For a nonnegative 8-bit index, the upper four bits identify the row and the lower four identify the column. This is the binary meaning of dividing a 16-column grid into rows.

Vector print order versus coordinate order

The notation q[15:0] prints bit 15 on the left, but our coordinate convention calls q[0] column 0. Keep that convention consistent in neighbors and traces. Reversing how a row is drawn does not change the recurrence if every coordinate mapping is reversed consistently.

Source: HDLBits Conwaylife. The problem specifies the row-to-vector mapping and a 16 x 16 grid.

BOUNDARIES / TOROIDAL NEIGHBORS

Wrap coordinates before flattening

A toroid connects top to bottom and left to right. Wrap the row and column independently, then turn each coordinate pair into a bit index.

```
up    = (row == 0)  ? 15 : row - 1;
down  = (row == 15) ? 0  : row + 1;
left  = (col == 0)  ? 15 : col - 1;
right = (col == 15) ? 0  : col + 1;
```

The eight neighbors of cell (0,0)

Direction	Wrapped coordinate	Read from old q
Up-left	(15,15)	q[255]
Up	(15,0)	q[240]
Up-right	(15,1)	q[241]
Left	(0,15)	q[15]
Right	(0,1)	q[1]
Down-left	(1,15)	q[31]
Down	(1,0)	q[16]
Down-right	(1,1)	q[17]

Why flat index + 1 can be wrong

At (0,15), the right neighbor is (0,0), index 0. Incrementing flat index 15 gives 16, which is (1,0), a different row. Even taking (index + 1) % 256 does not fix that row-boundary error. Wrap col within 0..15 first, while preserving row.

Do not count the center

The surrounding 3 x 3 square contains nine positions. The sum uses its eight outer positions and excludes q[row*16+col]. The center matters only for the two-neighbor survival rule. Every cell, including a corner, has eight distinct neighbors on this 16 x 16 toroid.

Source: HDLBits Conwaylife. The listed corner neighbors require wrapping on both axes.

RULES / PRESENT AND NEXT VALUES

Every cell reads the same old generation

q is the registered current grid. next_q is the combinational result of applying one generation of rules to that grid. All neighbor reads come from q.

Live neighbors	Current cell 0	Current cell 1	Assignment
0 or 1	0	0	next cell = 0
2	0	1	next cell = old cell
3	1	1	next cell = 1
4 through 8	0	0	next cell = 0

Why copy q when count is two?

Two neighbors preserve the center's existing state: a dead center stays dead and a live center stays live. Setting it unconditionally to 1 would incorrectly create a new cell. Copying q[row*16+col] expresses precisely that conditional survival.

A generation must use one consistent snapshot

Suppose an earlier loop iteration kills a neighbor. A later cell must still see that neighbor's old live value when computing this same generation. Reading partly updated next_q would mix two generations and make the result depend on loop traversal order. Reading only q avoids this contamination.

```
// Between rising edges:
next_q = F(q);

// At the rising edge:
if (load) q <= data;
else      q <= next_q;
```

next_q is a signal, not a function call

The combinational process already calculates next_q as q changes. At the edge, q <= next_q samples that calculated value and schedules the register update. After q changes, the combinational logic settles to the following generation's candidate. There is still only one registered advance per clock.

When a separate next value is optional

A single clocked process can instead assign each q bit with a nonblocking expression that reads only old q. In that version no next_q signal is needed. Keep the two-process form here because it clearly separates neighbor calculation from storage. Do not register next_q in another clocked block unless an additional pipeline cycle is intended.

Source: HDLBits Conwaylife. Rules and the one-generation-per-clock requirement.

COMPLETE RTL / ONE OWNER FOR q

The complete two-process solution

This is the editor solution with formatting and comments clarified. The combinational block assigns next_q; the rising-edge block alone assigns q.

```
module top_module (
    input clk,
    input load,
    input [255:0] data,
    output reg [255:0] q
);
    reg [255:0] next_q;
    integer row, col, up, down, left, right, count;

    always @(*) begin
        next_q = q;
        for (row = 0; row < 16; row = row + 1) begin
            for (col = 0; col < 16; col = col + 1) begin
                up    = (row == 0)  ? 15 : row - 1;
                down  = (row == 15) ? 0  : row + 1;
                left  = (col == 0)  ? 15 : col - 1;
                right = (col == 15) ? 0  : col + 1;
                count = q[up*16+left] + q[up*16+col]
                      + q[up*16+right] + q[row*16+left]
                      + q[row*16+right] + q[down*16+left]
                      + q[down*16+col] + q[down*16+right];
                if (count == 2)
                    next_q[row*16+col] = q[row*16+col];
                else if (count == 3)
                    next_q[row*16+col] = 1'b1;
                else
                    next_q[row*16+col] = 1'b0;
            end
        end
    end

    always @(posedge clk) begin
        if (load) q <= data;
        else      q <= next_q;
    end
endmodule
```

load has priority: the loading edge stores data unchanged. The first evolved generation is stored on the next rising edge with load low. There is no reset port in this interface; load supplies the initial grid.

EXECUTION / WIDTHS AND STORAGE

What the loops and assignments mean in hardware

Verilog source order describes how to calculate a result within a process. Clock edges determine when the stored grid changes. These are different kinds of sequencing.

The nested loops do not take 256 clocks

Each fixed-bound loop visits 16 coordinates during one evaluation of the combinational block. Together they describe the next-value logic for all 256 cells. Synthesis can expand these constant bounds into hardware; one cell is not processed per edge. A resource-shared multi-cycle design would need an explicit controller, address counter and different timing.

Use blocking assignments for the calculation

up, down, left and right must be available before the neighbor sum is evaluated. count must be available before the if statement reads it. Blocking = gives that immediate procedural calculation order. Nonblocking <= is used for the clocked q update so the old grid is sampled before the new grid becomes visible.

Why integer count does not overflow at eight

An integer is a 32-bit signed variable in Verilog. In this assignment, the addition is context-sized by that 32-bit destination, so the eight one-bit values are added at a sufficient width. All known inputs are 0 or 1, giving a result from 0 through 8. This code does not truncate each pairwise sum to one bit.

If refactoring the count into a function, concatenation or differently sized intermediate, check that expression's sizing rules again. Four unsigned bits are enough to represent 0..8. Explicitly zero-extending each bit to four bits before addition is a clear portable way to make the required width visible.

```
// Example of explicit width for one summand:
{3'b000, q[up*16+left]}
```

Defaults, latches and temporary variables

next_q = q provides a complete default before the loop. In this exact code, all 256 bits also receive an assignment through the complete if/else chain, so the default is redundant but safe. The count==2 branch still explicitly implements hold. Removing both default coverage and an assignment path would risk a latch in a combinational process.

The integer declarations do not themselves create flip-flops. Here the loop coordinates and count are procedural calculation temporaries. q is the clocked storage. The synthesized circuit still needs the logic for all cell results; using one source-level count variable does not promise one time-shared physical adder.

Connect this to the timer-series question

state <= next_state and q <= next_q both store a precomputed next value. In a timer, a test of state asks what state we are leaving; a test of next_state asks the chosen destination. Conway has no named controller states: q and next_q are the present and next complete board configurations. Neither name is a function invocation.

WORKED TRACE / REVISION CHECKS

Trace the boundary, then test the whole grid

The seed 256'h7 lights row 0, columns 0, 1 and 2. It is a useful boundary test because its next generation reaches row 15 across the top seam.

Stored generation	Live coordinates	Set bit indices
After load	(0,0), (0,1), (0,2)	0, 1, 2
After one advance	(15,1), (0,1), (1,1)	241, 1, 17
After two advances	(0,0), (0,1), (0,2)	0, 1, 2

Explain the first transition cell by cell

The center (0,1) sees two live neighbors and stays alive. The endpoints (0,0) and (0,2) each see only one and die. The dead cells directly above and below the center each see all three original live cells, so they are born. Above row 0 means row 15, giving the seam-crossing vertical line.

A loading edge is not an evolution edge

If load=1 and data=256'h7, that edge must leave q equal to 7, not the vertical line. While load remains high, each edge reloads the current data input. Once load becomes 0, each subsequent rising edge stores one generation. In simulation, inspect q after the nonblocking update has taken effect.

Useful independent checks

Case	Expected behavior / defect exposed
All zero	Stays zero; no spontaneous births
All one	Every cell has eight neighbors, so the next grid is zero
Single live cell	Dies after one advance
2 x 2 block	Each live cell has three neighbors; the block remains fixed
256'h7 blinker	Two-cycle oscillation across the top seam
Random loaded grids	Compare every output bit to an independently indexed model
All 512 local configurations	Both center values for all 256 eight-neighbor patterns

Final mental check

For any bit, recover row and col, wrap each neighbor coordinate, count eight old-q bits, apply the rule, and store the result on one edge. If a result differs, check coordinate wrapping before changing the rule. If the result is one generation late, check whether next_q was accidentally registered.

Source: HDLBits Conwaylife. The seed-7 oscillator is the problem's boundary-test hint.

LFSR: definition and essential vocabulary

LFSR means linear-feedback shift register: a bank of clocked bits whose next state is formed by shifting and feeding selected old bits back through XOR logic. In the autonomous binary XOR designs studied here, each next bit is a linear combination of old bits over GF(2), the two-element field.

What each word means

Term	Meaning in these exercises
Register / state	The n stored bits q[n-1:0]. A state is the complete bit pattern, not only the output bit.
Shift	Old values move to specified neighboring destinations on the active edge. The diagram determines direction.
Feedback / tap	A selected old bit participates in the next-state network. In HDLBits Galois drawings, numbered taps identify stages where feedback is injected.
Linear / GF(2)	XOR is addition modulo two: 1 XOR 1 = 0, with no carry. A copied bit is also a linear expression.
Seed	The starting state established by reset or parallel load. It selects where a deterministic trajectory begins.
Period	The number of updates before the sequence repeats. An n-bit maximal XOR LFSR has a nonzero-state period of 2^n - 1.
Lockup	A state that maps to itself. With only XOR/copy feedback, all-zero state remains all zero.

A clock edge computes a new state

For Lfsr5, q=00001 gives outgoing old q[0]=1. The base shift is 10000; the tap changes destination q[2] to old q[3] XOR old q[0] = 1. The next state is 10100. Both steps describe one edge, not two clock cycles.

Do not interpret the 5-bit word as a binary count. It progresses 01, 14, 0A, 05, ... in hexadecimal, not 01, 02, 03, 04. Reapplying the same seed and inputs reproduces the same sequence.

Source convention: Philip Koopman, Maximal Length LFSR Feedback Terms, right-shift recurrence. The worked five-bit calculation is derived from the saved HDLBits solution.

Taps, feedback forms and maximum period

Fibonacci and Galois describe different placements of the feedback XORs. In the Fibonacci form, selected stages combine into feedback entering one end. In the Galois form, the outgoing bit is distributed into selected internal stage updates. Equivalent polynomials can require a state/phase mapping; copying the same seed bits does not guarantee identical cycle-by-cycle register words.

Translate the convention before copying a tap list

For the HDLBits right-shifting five-bit Galois circuit, tap positions 5 and 3 correspond to destinations q[4] and q[2]. Position k becomes index k-1. The lower tap uses the ordinary shifted source q[k] XOR outgoing q[0]; the top destination receives q[0]. This rule belongs to this drawing convention, not every published LFSR diagram.

```
next_q = (q >> 1) ^ (q[0] ? 5'b10100 : 5'b00000);
// mask 10100 injects feedback into destinations 4 and 2
```

Why not every tap choice gives maximum length

A nonzero seed alone is insufficient. If the five-bit mask is 10000, the update merely rotates: 00001 -> 10000 -> 01000 -> 00100 -> 00010 -> 00001. Its period from this seed is only five. The exercise mask 10100 instead visits all 31 nonzero states before returning to 00001.

A feedback polynomial records the recurrence algebraically; its coefficients are bits and arithmetic is over GF(2). A degree-n primitive polynomial is irreducible and gives multiplicative order 2^n - 1. The implementation must use the matching orientation and feedback convention. Reversing bit direction can introduce the reciprocal polynomial.

Design	Nonzero period	Reference transition
Mt2015 three-bit circuit	7	001 -> 010
HDLBits five-bit Galois	31	00001 -> 10100
HDLBits 32-bit Galois	4,294,967,295	00000001 -> 80200003 (hex)

The short periods were exhaustively enumerated. The 32-bit period is supported by the primitive-order calculation, not by simulating over four billion clock edges. Loading zero into these XOR circuits gives period one instead.

XOR and XNOR lockup rules differ

The conventional XNOR-feedback construction in XAPP052 excludes the all-ones state; these XOR exercises exclude zero. Do not swap operators or reset values based only on a tap table.

References: Peter Alfke, Xilinx XAPP052, pp. 5-6 (tap-numbering and XNOR convention); Koopman (right-shift masks). Equations and the five-state counterexample are derived here.

Applications, limits and clocked-state revision

What it is useful for

An LFSR provides repeatable test patterns and long periodic bit sequences with simple XOR-and-register logic. It can act as a sequence counter when the numerical binary order is irrelevant. A compact state generator is not a substitute for a binary counter when downstream logic needs the actual arithmetic count.

PRBS means pseudo-random binary sequence. The word pseudo matters: a fixed recurrence is predictable from its state. A long period does not make a bare LFSR a secure random-number generator. Its equations are linear, so it should not be used by itself to generate secret keys or security tokens.

Scramblers and CRC circuits use related shift/XOR structures, but commonly mix incoming data into the recurrence. A CRC register stores a remainder under a specified polynomial and bit-order convention. An autonomous HDLBits LFSR advancing without message data is not, by itself, a complete CRC implementation.

Current state and next state: the shared coding rule

Question	Decision
What is q / state?	The value stored before the active edge. Use it to perform work belonging to the current phase or generation.
What is next_q / next_state?	The combinationally computed candidate for storage. It is a signal, not a function call and not an extra clock cycle.
Must I declare next_q?	No. A single clocked block can write q <= expression_of_old_q. A separate complete combinational block can instead calculate next_q, then store q <= next_q.
When inspect next_state in a clocked FSM block?	Only for an intended response to the chosen destination. To act once on entry, test state != TARGET && next_state == TARGET.
How avoid a partly updated state?	Use nonblocking <= for clocked registers. All right-hand sides in the shown process read old state; a later overlapping assignment replaces the scheduled destination, not the old source.

Quick checks before trusting a new LFSR

Name the shift direction and feedback bit; translate numbered taps; match reset/load priority; predict the first post-seed transition; test zero lockup deliberately; then check the period with a method appropriate to the width. An unknown simulation state is not a random seed: establish a known nonzero state first.

Reference: XAPP052, pp. 4 and 6, deterministic sequences and LFSR counters. The security warning follows from the explicit linear recurrence; the coding recap applies the old-value rules already traced in the timer, LFSR and Conway chapters.